

# Formulação da Lei de Controle Cinemático para um Manipulador 2-DOF (Mínimos Quadrados via Pseudoinversa)

Gemini

29 de outubro de 2025

## 1 Objetivo do Controle

O objetivo do código fornecido é implementar um controle de posição em malha fechada. Dado um ponto de referência (alvo)  $\mathbf{x}_d \in \mathbb{R}^2$  no espaço cartesiano, a lei de controle deve calcular iterativamente as velocidades de junta  $\dot{\mathbf{q}} \in \mathbb{R}^2$  que fazem com que a posição do efetuador final do robô,  $\mathbf{x}$ , convirja para  $\mathbf{x}_d$ .

A estratégia utilizada é o **Controle de Movimento em Taxa Resolvida** (Resolved-Rate Motion Control), que opera no nível de velocidade.

## 2 Modelo Cinemático

### 2.1 Cinemática Direta (Posição)

A relação não linear entre as posições das juntas  $\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$  e a posição cartesiana do efetuador final  $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$  é dada pela função de cinemática direta,  $f$ :

$$\mathbf{x} = f(\mathbf{q})$$

No código, esta função é `forward_kinematics(q1, q2)`.

### 2.2 Cinemática Diferencial (Velocidade)

Para relacionar as velocidades, derivamos a equação acima em relação ao tempo, usando a regra da cadeia:

$$\dot{\mathbf{x}} = \frac{d}{dt}f(\mathbf{q}) = \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \frac{d\mathbf{q}}{dt}$$

Isso nos dá a relação fundamental da cinemática diferencial, que envolve a matriz Jacobiana  $\mathbf{J}(\mathbf{q})$ :

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{1}$$

Onde  $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{2 \times 2}$  é a Jacobiana (calculada pela função `get_jacobian`) que mapeia as velocidades de junta ( $\dot{\mathbf{q}}$ ) para as velocidades cartesianas ( $\dot{\mathbf{x}}$ ).

### 3 Formulação da Lei de Controle

A lei de controle implementada é uma combinação de duas etapas, executadas em cada passo da simulação.

#### 3.1 Etapa 1: Controlador Proporcional no Espaço de Tarefa

Primeiro, definimos o erro de posição no espaço de tarefa (cartesiano):

$$\mathbf{e} = \mathbf{x}_d - \mathbf{x} \quad (2)$$

Isso corresponde à linha de código: `error = target_pos_robot - current_pos`.

Para fazer o robô se mover em direção ao alvo, a forma mais simples de controle é definir uma **velocidade cartesiana desejada**,  $\dot{\mathbf{x}}_d$ , que seja *proporcional* ao erro. Esta é a lei de um Controlador Proporcional (P):

$$\dot{\mathbf{x}}_d = K_p \mathbf{e} \quad (3)$$

Onde  $K_p$  é um ganho escalar positivo (definido como `KP` no código). Esta lei garante que, se conseguirmos impor  $\dot{\mathbf{x}} = \dot{\mathbf{x}}_d$ , o erro  $\mathbf{e}(t)$  convergirá exponencialmente para zero.

Isso corresponde à linha de código: `ee_velocity = error * KP`.

#### 3.2 Etapa 2: Solução de Cinemática Inversa por Mínimos Quadrados

Agora, temos uma velocidade cartesiana desejada ( $\dot{\mathbf{x}}_d$ ) e queremos encontrar as velocidades de junta ( $\dot{\mathbf{q}}$ ) que a produzem. Substituímos  $\dot{\mathbf{x}}$  por  $\dot{\mathbf{x}}_d$  na Equação (1):

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \dot{\mathbf{x}}_d$$

Nosso objetivo é resolver para  $\dot{\mathbf{q}}$ . Este é um sistema de equações lineares. O método de **Mínimos Quadrados**, solicitado na formulação, busca encontrar o vetor  $\dot{\mathbf{q}}$  que minimiza a norma do erro ao quadrado:

$$\min_{\dot{\mathbf{q}}} \|\mathbf{J}\dot{\mathbf{q}} - \dot{\mathbf{x}}_d\|^2 \quad (4)$$

A solução analítica para este problema de otimização é dada pela **Pseudoinversa de Moore-Penrose** da Jacobiana, denotada por  $\mathbf{J}^+$ :

$$\dot{\mathbf{q}} = \mathbf{J}^+ \dot{\mathbf{x}}_d \quad (5)$$

No código, esta solução de mínimos quadrados é calculada diretamente pela função `np.linalg.pinv()`:

- `J_pinv = np.linalg.pinv(J)`

Esta função calcula  $\mathbf{J}^+$  de forma numericamente robusta.

## 4 Implementação Discreta e Resumo

Combinando as Etapas 1 e 2, a lei de controle completa para a velocidade da junta é:

$$\dot{\mathbf{q}} = \mathbf{J}^+(K_p(\mathbf{x}_d - \mathbf{x}))$$

No código, esta lei é implementada de forma iterativa (discreta no tempo), onde  $k$  é o passo de tempo atual:

1. **Calcular Posição Atual:**  $\mathbf{x}_k = f(\mathbf{q}_k)$
2. **Calcular Erro:**  $\mathbf{e}_k = \mathbf{x}_d - \mathbf{x}_k$
3. **Calcular Velocidade Cartesiana Desejada:**  $\dot{\mathbf{x}}_d = K_p \mathbf{e}_k$
4. **Calcular Jacobiana:**  $\mathbf{J}_k = \mathbf{J}(\mathbf{q}_k)$
5. **Resolver Mínimos Quadrados (Pseudoinversa):**  $\dot{\mathbf{q}}_k = \mathbf{J}_k^+ \dot{\mathbf{x}}_d$   
(dq = J\_pinv @ ee\_velocity)
6. **Integrar (Método de Euler):**  $\mathbf{q}_{k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_k \cdot \Delta t$   
(q\_current += dq \* DT)

Este ciclo se repete até que a norma do erro,  $\|\mathbf{e}_k\|$ , seja menor que a tolerância definida (TOLERANCE).

### 4.1 Limitação

Esta formulação de Mínimos Quadrados pura, embora matematicamente correta, é suscetível a instabilidades numéricas quando o robô se aproxima de uma **singularidade** (por exemplo, braço totalmente esticado), onde  $\mathbf{J}$  perde posto. Nesses pontos, os valores em  $\mathbf{J}^+$  tendem ao infinito, causando "explosões" nas velocidades das juntas  $\dot{\mathbf{q}}$ .